ASSIGNMENT 2

Problem Definition:

Large Language Models (LLMs) are trained on general data and can perform many tasks, but they do not always work well for specific tasks like code generation. Their outputs may be incorrect, incomplete, or not properly structured. The problem in this project is to improve the performance of a pre-trained model for generating code. The base model often gives generic or partially correct answers.

To solve this, the model is fine-tuned using a custom dataset containing coding-related instructions and their correct outputs. This helps the model learn how to generate better and more accurate code.

The objective of this project is to build a code generation assistant by fine-tuning a language model and then evaluating its performance.

Dataset creation methodology:

In this project, a custom dataset was created for fine-tuning the language model. The dataset contains around 1100 examples of coding-related tasks. Each example in the dataset has two parts: an input and an output. The input contains a coding instruction, and the output contains the correct code for that instruction.

The dataset was created using synthetic data generation, where different coding problems such as writing functions, fixing bugs, and creating APIs were included. This helps the model learn different types of coding tasks.

The dataset was stored in JSON format and later converted into JSONL format for training. It was then divided into three parts: training set, validation set, and test set.

Before training, the data was formatted into an instruction-response format. This formatting helps the model understand how to respond correctly to instructions.

One such example:

```
    {"input": "Write a Python function to check if a string is a
palindrome, ignoring case and non-alphanumeric characters.",

    "output": "import re\n\ndef is_palindrome(s):\n    # Remove
non-alphanumeric characters and convert to lowercase\n    cleaned =
re.sub(r'[^a-zA-Z0-9]', '', s).lower()\n  }
```

Model Architecture and Fine-Tuning Method:

In this project, the TinyLlama 1.1B model was selected as the base model for fine-tuning. It is an open-source, lightweight large language model that is suitable for tasks like code generation and can run efficiently on limited computational resources.

Since training the full model requires high computational power, a parameter-efficient fine-tuning technique called QLoRA (Quantized Low-Rank Adaptation) was used. QLoRA allows the model to be fine-tuned by updating only a small set of additional parameters instead of modifying all the model weights. This significantly reduces memory usage and makes training feasible on platforms like Google Colab.

The fine-tuning process was implemented using the HuggingFace Transformers library along with the PEFT (Parameter Efficient Fine-Tuning)

framework. The model was loaded in 4-bit quantized form to further optimize memory usage.

During training, the model was provided with instruction-response pairs from the custom dataset. The input was formatted in a structured way so that the model learns how to generate correct code responses based on given instructions.

The training was performed in Google Colab using GPU support, which helped in faster computation and efficient fine-tuning of the model.

```
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
import torch

model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

# Quantization config (QLoRA)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4"
)

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token  # IMPORTANT

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)
```

```
Loading weights: 100%|| 201/201 [00:01<00:00, 287.68it/s, Materializing param=model.norm.weight]
```

Fine-tuning of TinyLlama model using QLoRA with HuggingFace Transformers in Google Colab

Training Configuration:

The training of the model was configured carefully to balance performance and computational efficiency. Since the model was fine-tuned using QLoRA, a low-resource setup was sufficient for the training process.

The training was performed in Google Colab using GPU acceleration to speed up computations. The dataset was tokenized and padded to a fixed maximum sequence length to ensure consistent input size.

The key training parameters used are as follows:

- Number of epochs: 2
- Batch size: 1 (due to memory constraints)
- Learning rate: 2e-4
- Maximum sequence length: 512 tokens
- Quantization: 4-bit (to reduce memory usage)

The HuggingFace Trainer API was used to manage the training process, including loss computation and optimization. Labels were set equal to input IDs to enable causal language modeling.

The model was trained on the training dataset and evaluated on the validation dataset during the process. After training, only the adapter weights were saved instead of the full model, as QLoRA updates only a small portion of the parameters.

This configuration ensured that the model could be fine-tuned efficiently while still learning meaningful patterns from the dataset.

```python
def format_example(example):
    return {
        "text": f"### Instruction:\n{example['input']}\n\n### Response:\n{example['output']}"
    }

dataset = dataset.map(format_example)
```

```python
def tokenize(example):
    tokens = tokenizer(
        example["text"],
        truncation=True,
        padding="max_length",
        max_length=512
    )

    tokens["labels"] = tokens["input_ids"].copy()

    return tokens

dataset = dataset.map(
    tokenize,
    batched=True,
    remove_columns=dataset["train"].column_names
)
```

```
Map: 100% [=================================] 932/932 [00:00<00:00, 1392.68 examples/s]
Map: 100% [=================================] 116/116 [00:00<00:00, 1092.45 examples/s]
```

Dataset preprocessing and tokenization pipeline showing instruction-response formatting and conversion into input IDs and labels for model training.

```python
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"]
)

trainer.train()
```

[1864/1864 10:27, Epoch 2/2]

Epoch	Training Loss	Validation Loss
1	0.143081	0.155061
2	0.149906	0.148948

```
TrainOutput(global_step=1864, training_loss=0.24032179592377126, metrics={'train_runtime': 629.1035, 'train_samples_per_second': 2.963, 'train_steps_per_second': 2.963, 'total_flos': 5930283090051072.0, 'train_loss': 0.24032179592377126, 'epoch': 2.0})
```

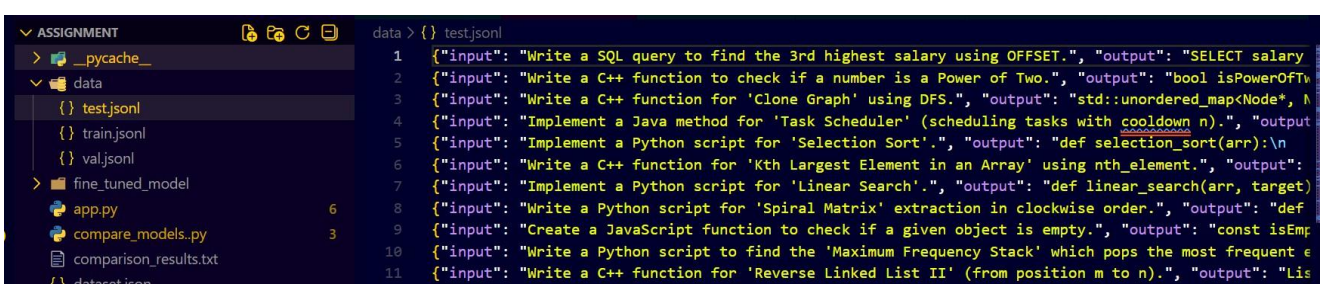
Model training using HuggingFace Trainer showing loss reduction across epochs

Tools and Technologies Used:

- **Python:** Used as the main programming language for data preprocessing, model training, evaluation, and backend development.
- **HuggingFace Transformers:** Used to load the pre-trained TinyLlama model and perform fine-tuning and inference.
- **PEFT (Parameter Efficient Fine-Tuning):** Used to implement QLoRA, allowing efficient fine-tuning by updating only a small number of parameters.
- **Google Colab:** Used for training the model with GPU support, enabling faster and efficient computation.
- **FastAPI:** Used to build the backend API, allowing users to send prompts and receive generated code through the `/generate` endpoint.
- **HTML, CSS, JavaScript:** Used to develop a simple and interactive frontend UI for real-time code generation.

Workflow / Methodology:

- **Dataset Creation:** A custom dataset of around 1100 examples was created, containing coding-related instructions and their corresponding outputs. The dataset includes tasks such as code generation, bug fixing, and API development.
- **Data Preprocessing & Splitting** : The dataset was cleaned and formatted into an instruction-response structure using “# Instruction” and “# Response”. The dataset was divided into training, validation, and test sets to ensure proper training and evaluation.



- **Model Fine-Tuning Process**: The TinyLlama 1.1B model was selected and fine-tuned using the QLoRA technique with the help of the PEFT library. The dataset was tokenized using the model's tokenizer with padding and truncation applied. Training was performed using the HuggingFace Trainer API in Google Colab with GPU support, where only adapter weights were updated instead of the full model.
- **Model Saving and Usage**: After training, the fine-tuned adapter model was saved and later downloaded from Google Colab. It was then loaded in a local environment (VS Code) for inference.

```
model.save_pretrained("fine_tuned_model")
tokenizer.save_pretrained("fine_tuned_model")
```

```
('fine_tuned_model/tokenizer_config.json',
 'fine_tuned_model/chat_template.jinja',
 'fine_tuned_model/tokenizer.json')
```

```
import os
print(os.listdir())
```

```
['.config', 'results', 'train.jsonl', 'val.jsonl', 'fine_tuned_model', '.ipynb_checkpoints', 'sample_data']
```

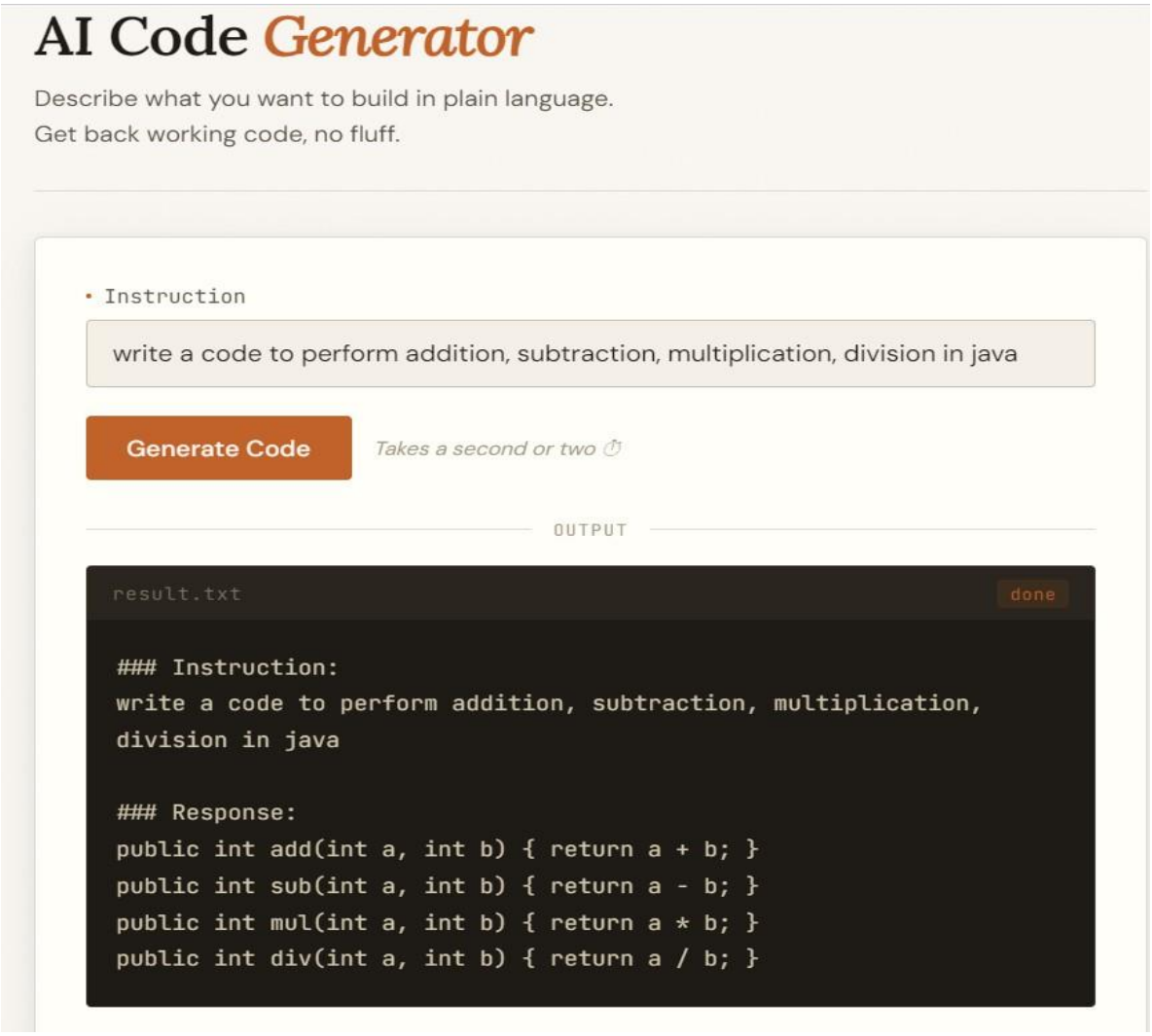
```
!zip -r model.zip fine_tuned_model
```

```
  adding: fine_tuned_model/ (stored 0%)
  adding: fine_tuned_model/tokenizer_config.json (deflated 46%)
  adding: fine_tuned_model/tokenizer.json (deflated 85%)
  adding: fine_tuned_model/adapter_config.json (deflated 57%)
  adding: fine_tuned_model/adapter_model.safetensors (deflated 8%)
  adding: fine_tuned_model/chat_template.jinja (deflated 60%)
  adding: fine_tuned_model/README.md (deflated 66%)
```

```
from google.colab import files
files.download("model.zip")
```

- **Application Setup**: The model was connected to a FastAPI backend and a simple frontend UI was created to generate code responses in real time.

```
o` for more details.
Both `max_new_tokens` (=120) and `max_length`(=2048) seem to have been set. `max_new_tokens` will take precedence.
refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/tex
ation)
INFO:     127.0.0.1:64543 - "POST /generate HTTP/1.1" 200 OK
INFO:     127.0.0.1:58536 - "OPTIONS /generate HTTP/1.1" 200 OK
Both `max_new_tokens` (=120) and `max_length`(=2048) seem to have been set. `max_new_tokens` will take precedence.
refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/tex
ation)
INFO:     127.0.0.1:58536 - "POST /generate HTTP/1.1" 200 OK
```



Evaluation Results:

The performance of the fine-tuned model was evaluated using standard metrics such as BLEU, ROUGE, and Accuracy.

The results obtained are as follows:

- BLEU Score: 0.15
- ROUGE Score: Approximately 0.42

The BLEU score is relatively low, which indicates that the generated outputs do not exactly match the expected reference outputs. This is expected in code generation tasks, as there can be multiple correct ways to write the same code.

The ROUGE score is moderate, showing that the generated outputs have a reasonable overlap in terms of keywords and structure with the expected outputs. This indicates that the model is able to understand the task and generate relevant responses.

Overall, the evaluation results show that the fine-tuned model is able to generate meaningful and relevant code, although it may not always match the expected output exactly.

```
 🔹 PROMPT: Write a Python function to find max element in list
 🔹 GENERATED: ### Instruction:
Write a Python function to find max element in list

### Response:
def max_element(nums):
    max_ = nums[0]
    for n in nums[1:]:
        if n > max_:
            max_ = n
    return max_
 🔹 EXPECTED: def max_element(lst): return max(lst)

📊 FINAL RESULTS
BLEU: {'bleu': 0.15003207874129063, 'precisions': [0.25102880658436216, 0.18487394957983194, 0.12446351931330472, 0.087719
29824561403], 'brevity_penalty': 1.0, 'length_ratio': 3.4225352112676055, 'translation_length': 243, 'reference_length': 7
1}
ROUGE: {'rouge1': np.float64(0.41880804953560374), 'rouge2': np.float64(0.3043650793650794), 'rougeL': np.float64(0.389830
36190882886), 'rougeLsum': np.float64(0.3860656560264759)}
```

Additionally, a comparison between the base model and the fine-tuned model was performed. The fine-tuned model generally produced more structured and task-specific responses, demonstrating improvement over the base model.

```
PROMPT:
Write optimized Python code to remove duplicates from a list

BASE MODEL OUTPUT:
### Instruction:
Write optimized Python code to remove duplicates from a list

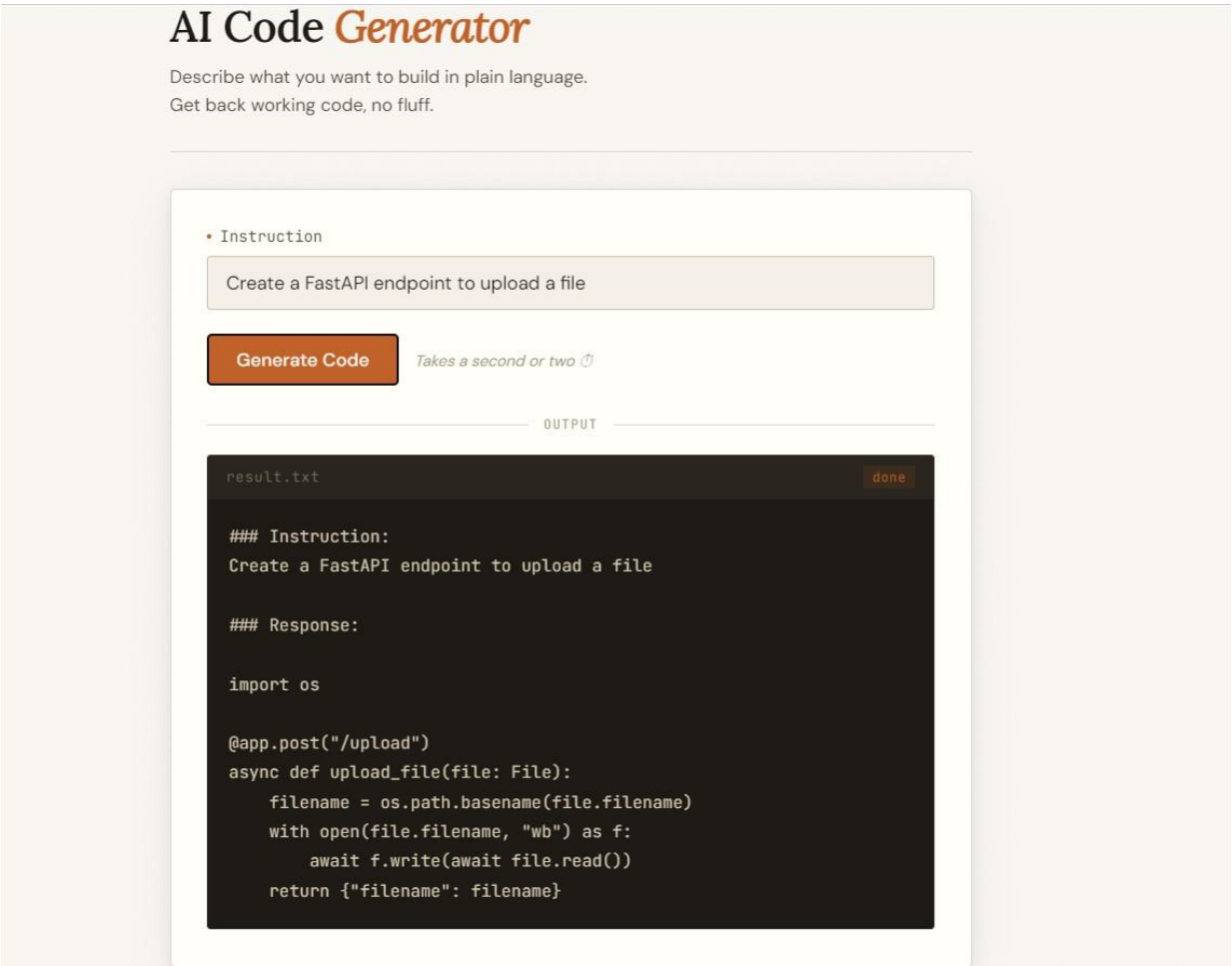
### Response:
```python
def remove_duplicates(lst):
    # Create a new list to store the unique elements
    unique_lst = []

    # Loop through the original list and add each element to the unique list
    for elem in lst:
        if elem not in unique_lst:
            unique_lst.append(elem)

    # Return the unique list
    return unique_lst

# Example usage
lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
unique_lst = remove_duplicates(lst)
print(unique_lst)
```

FINE-TUNED MODEL OUTPUT:
Instruction:
Write optimized Python code to remove duplicates from a list

Response:
def removeDuplicates(lst):
    if not lst: return []
    n = len(lst)
    for i in range(n):
        if lst[i] != lst[i+1]: lst[i+1] = lst[i]
    return lst[:n]
```

Example outputs were also analyzed to understand the strengths and limitations of the model. Some outputs were correct, while others were partially correct or incorrect, especially in complex logic cases.